Создай установщик через pip setup py

Да. Добавим установку через `pip` с помощью `setup.py`. Проект можно будет установить командой:

```
pip install .
```

И запускать как консольную команду:

```
rrm-run
```

Обновлённая структура

```
responsibility-response-model/
├── rrm/
│   ├── __init__.py
│   ├── model.py
│   ├── scenarios.py
│   ├── visualization.py
│   └── cli.py
├── tests/
│   ├── __init__.py
│   ├── test_model.py
│   └── test_scenarios.py
├── .gitignore
├── requirements.txt
├── setup.py
├── README.md
└── LICENSE
```

`setup.py` должен лежать в корне проекта; `pip install .` работает с проектами, содержащими `setup.py` или `pyproject.toml`.[1] [2] [3]

Файл: setup.py

Сохрани в корне проекта:

```
# -*- coding: utf-8 -*-

from setuptools import setup, find_packages

setup(
```

```
    name="responsibility-response-model",
    version="1.0.0",
    description=(
        "Универсальная модель ответственности: "
        "оценка пользы, риска, задержки помощи и компенсации."
    ),
    long_description=open("README.md", encoding="utf-8").read(),
    long_description_content_type="text/markdown",
    author="Your Name",
    author_email="your@email.com",
    url="https://github.com/YOUR_USERNAME/responsibility-response-model",
    license="MIT",

    packages=find_packages(include=["rrm", "rrm.*"]),

    python_requires=">=3.9",

    install_requires=[
        "numpy",
        "matplotlib",
        "pillow",
    ],

    extras_require={
        "dev": [
            "pytest",
        ]
    },

    entry_points={
        "console_scripts": [
            "rrm-run=rrm.cli:main",
        ]
    },

    classifiers=[
        "Programming Language :: Python :: 3",
        "Programming Language :: Python :: 3.9",
        "Programming Language :: Python :: 3.10",
        "Programming Language :: Python :: 3.11",
        "Programming Language :: Python :: 3.12",
        "License :: OSI Approved :: MIT License",
        "Operating System :: OS Independent",
        "Topic :: Scientific/Engineering",
        "Topic :: Sociology",
    ],
)
```

`install_requires` указывает зависимости, которые `pip` установит вместе с пакетом; `entry_points` создаёт консольную команду `rrm-run`.[4] [5] [6]

Файл: rrm/cli.py

Сохрани как `rrm/cli.py`:

```
# -*- coding: utf-8 -*-
"""Консольный запуск модели."""

import importlib.util
import shutil
import subprocess
import sys

REQUIRED_PACKAGES = ["numpy", "matplotlib", "pillow"]


def check_python_packages():
    missing = []

    for package in REQUIRED_PACKAGES:
        spec = importlib.util.find_spec(package)

        if spec is None:
            missing.append(package)
            print(f"[НЕТ] {package} не установлен.")
        else:
            print(f"[ОК] {package} установлен.")

    if missing:
        answer = input(
            f"\nДля работы нужны: {', '.join(missing)}.\n"
            "Установить их сейчас через pip? [y/n]: "
        ).strip().lower()

        if answer in ("y", "yes", "д", "да"):
            for package in missing:
                print(f"Устанавливаю {package}...")
                subprocess.check_call(
                    [sys.executable, "-m", "pip", "install", package]
                )
        else:
            print("\nУстановка отменена.")
            sys.exit(1)

    for package in REQUIRED_PACKAGES:
        if importlib.util.find_spec(package) is None:
            print(f"Ошибка: {package} не удалось установить.")
            sys.exit(1)

    print("\nPython-библиотеки готовы.\n")


def check_ffmpeg():
    ffmpeg_path = shutil.which("ffmpeg")

    if ffmpeg_path is None:
        print("[НЕТ] ffmpeg не найден. Будет создан GIF.")
```

```
        return False

    print(f"[OK] ffmpeg найден: {ffmpeg_path}")
    return True


def main():
    check_python_packages()
    ffmpeg_available = check_ffmpeg()

    from rrm.visualization import (
        plot_delay_curves,
        plot_scenario_comparison,
        plot_reaction_speed,
        plot_model_principle,
        create_animation
    )

    print("Запуск Responsibility Response Model...\n")

    plot_delay_curves()
    print("Создан график: 1_delay_coefficient.png")

    plot_scenario_comparison()
    print("Создан график: 2_system_comparison.png")

    plot_reaction_speed()
    print("Создан график: 3_reaction_speed.png")

    plot_model_principle()
    print("Создан график: 4_model_principle.png")

    create_animation(export_mp4=True, ffmpeg_available=ffmpeg_available)

    print("\nВсе файлы созданы успешно.")


if __name__ == "__main__":
    main()
```

Файл: rrm/init.py

```
# -*- coding: utf-8 -*-

from rrm.model import delay_coefficient, model_value, support_function
from rrm.scenarios import evaluate_scenarios

__version__ = "1.0.0"
```

Обновлённый requirements.txt

```
numpy
matplotlib
pillow
pytest
```

Как установить проект

Внутри папки проекта:

```
python -m venv .venv
source .venv/bin/activate      # Linux/macOS
.venv\Scripts\activate         # Windows

pip install -e .
```

Флаг `-e .` устанавливает проект в **редактируемом режиме**: изменения в коде сразу видны без повторной установки.

Для обычной установки:

```
pip install .
```

`pip install .` автоматически находит `setup.py` в текущей папке и устанавливает пакет вместе с зависимостями. [2] [3] [7]

Как запустить после установки

После установки станет доступна команда:

```
rrm-run
```

Она:

1. Проверит `numpy`, `matplotlib`, `pillow`.
2. Проверит наличие `ffmpeg`.
3. Создаст четыре PNG-графика.
4. Создаст `model_delay_animation.mp4`, если есть `ffmpeg`.
5. Создаст `model_delay_animation.gif`, если `ffmpeg` нет.

Как запустить тесты

```
pytest
```

Или подробнее:

```
pytest -v
```

Как опубликовать на PyPI

Если захочешь сделать пакет доступным через:

```
pip install responsibility-response-model
```

нужно собрать дистрибутивы:

```
python -m pip install --upgrade build
python -m build
```

После этого в папке появятся:

```
dist/
├── responsibility_response_model-1.0.0.tar.gz
└── responsibility_response_model-1.0.0-py3-none-any.whl
```

Затем их можно загрузить на PyPI через `twine`.

Важное замечание

Современная документация Python Packaging Authority рекомендует для новых проектов использовать `pyproject.toml`, а `python setup.py install` считать устаревшим подходом. Однако `setup.py` по-прежнему поддерживается `pip` и `setuptools`, поэтому для твоего запроса он подходит. [8] [9] [10]

Финальная формула

$$\boxed{V(T) = B(T) - R(T) + \int_0^T e^{-\lambda t} S(t) dt}$$

Финальный принцип

> Любая система ответственна не только за решение, но и за скорость и качеств

Теперь проект можно:

- установить через `pip install .`;

- запускать командой `rrm-run`;
- тестировать через `pytest`;
- публиковать на GitHub;
- в будущем опубликовать на PyPI. [1] [2] [4]

⁂

1. https://packaging.python.org/en/latest/guides/distributing-packages-using-setuptools/
2. https://pip.pypa.io/en/stable/cli/pip_install/
3. https://pip.pypa.io/en/latest/cli/pip_install/
4. https://setuptools.pypa.io/en/latest/userguide/dependency_management.html
5. https://setuptools.pypa.io/en/stable/references/keywords.html
6. https://packaging.python.org/en/latest/guides/distributing-packages-using-setuptools/?highlight=bdist_wheel
7. https://pip.pypa.io/en/stable/cli/pip_install/?highlight=find-links
8. https://setuptools.pypa.io/en/latest/userguide/quickstart.html
9. https://packaging.python.org/en/latest/tutorials/packaging-projects/?highlight=python_requires
10. https://packaging.python.org/en/latest/guides/tool-recommendations/
11. https://docs.python.org/uk/3.10/install/index.html
12. https://setuptools.pypa.io/en/stable/_sources/userguide/declarative_config.rst.txt
13. https://pip.pypa.io/en/stable/cli/pip_install/?highlight= Local project installs
14. https://docs.fossa.com/docs/project-setup/supported-languages/python/setuptools
15. https://github.com/pypa/setuptools/blob/52aacd5b276fedd6849c3a648a0014f5da563e93/docs/setuptools.txt